

DEEP NEURAL NETWORKS (AIML ZG565) · COMPANION READER

Linear Neural Networks for Classification

Sigmoid, softmax, cross-entropy, SGD, and the full classification pipeline

Session 4 (17 May 2026) · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give the formulas; this reader gives the *why*. Every concept is expanded into plain-English prose and every slide example is worked step by step. Colour code: **blue** intuition; **amber** everyday picture; **green** worked example; **red** watch out; **purple** key takeaway.

1 What this session covers

Session 3 trained a single neuron to predict a *number*. Session 4 asks: what if the answer is a *category* instead? The change is surprisingly small: swap the identity activation for a **sigmoid** (binary case) or a **softmax** (multi-class case), swap the MSE loss for **cross-entropy**, and choose the right gradient update. Everything else — the four-ingredient framework, design matrices, gradient descent — carries over unchanged.

The session covers binary classification (logistic regression), then extends to multi-class (softmax regression), introduces **stochastic** and **mini-batch** gradient descent, and ends with the full suite of classification metrics: confusion matrix, accuracy, precision, recall, F1, and top- K accuracy.

2 What is classification?

Classification is supervised learning where the output is a discrete category. The input is still a feature vector $\mathbf{x} \in \mathbb{R}^d$; the output is a class label $y \in \{1, 2, \dots, K\}$ from a fixed set of K classes.

The intuition

Classification answers “which box does this belong in?” Regression answers “how big is this number?” Both use a feature vector as input; only the shape of the output differs.

Three flavours:

- **Binary:** $K = 2$, e.g. spam vs. not-spam, disease present vs. absent.
- **Multi-class:** $K > 2$, one correct class per example, e.g. digit 0–9, dog breed.
- **Multi-label:** each example can belong to several classes simultaneously, e.g. a film tagged as both action and comedy.

Where this shows up in ML. Every image classifier, spam filter, disease detector, and language model token predictor is a classification system. The softmax layer at the top of GPT is multi-class classification over a vocabulary of $\approx 50,000$ words.

✓ Key takeaway

Classification = supervised learning with a discrete output. Binary ($K=2$): sigmoid. Multi-class ($K>2$): softmax.

3 Why not use linear regression for classification?

If we tried to use the linear regression model with identity activation and MSE loss for binary classification (labelling spam as 1, not-spam as 0), two problems arise:

1. The predicted value $\hat{y} = \mathbf{w}^\top \mathbf{x}$ can be any real number, but probabilities must lie in $[0, 1]$.
2. MSE treats the error of predicting 5.3 for a 0/1 label the same way it would for a continuous target. Outlier inputs can push the decision boundary far from where it should be.

The fix is to squash the output through a function that maps $\mathbb{R} \rightarrow (0, 1)$: the **sigmoid**.

4 Binary classification: logistic regression

4.1 The sigmoid activation

The **sigmoid** (or logistic) function maps any real number to the open interval $(0, 1)$, making the output interpretable as a probability:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (1)$$

Named symbols: z is the pre-activation (the weighted sum); e is Euler's number (≈ 2.718); σ (sigma) is the standard symbol for the sigmoid. Key values: $\sigma(0) = 0.5$ (maximum uncertainty), $\sigma(z) \rightarrow 1$ as $z \rightarrow +\infty$ (confident class 1), $\sigma(z) \rightarrow 0$ as $z \rightarrow -\infty$ (confident class 0).

🧠 The intuition

The sigmoid is a “soft step”: it turns the perceptron's hard 0/1 snap into a smooth ramp. Large positive z means “very probably class 1”; large negative z means “very probably class 0”; $z = 0$ means “I have no idea”.

★ Everyday picture

A weather app shows “80% chance of rain”. Behind the scenes it computed some weighted sum z of temperature, humidity, and pressure, then passed z through the sigmoid to get 0.80. The threshold 0.5 is the boundary: above it we carry an umbrella, below it we don't.

The derivative has an elegant closed form:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)). \quad (2)$$

This self-referential form means backpropagation only needs $\sigma(z)$ (already computed in the forward pass) to compute the gradient — efficient.

Where this shows up in ML. The sigmoid is the output activation for binary classification. It is also used inside recurrent networks (LSTM gates) and as the attention gate in transformers — anywhere you need a number controlled to $[0, 1]$.

Sigmoid $\sigma(z) = 1/(1+e^{-z})$: squashes any real number into (0,1)

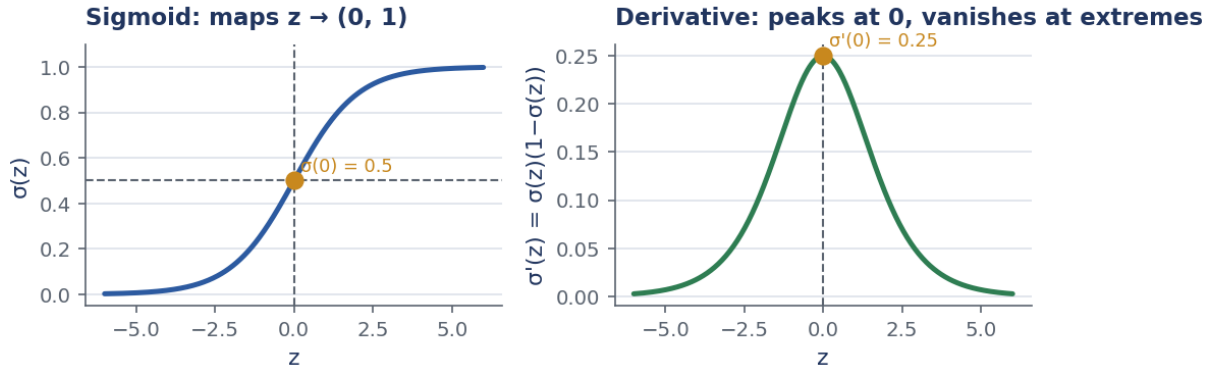


Figure 1: How to read it: left panel is $\sigma(z)$ — note how it asymptotes to 0 and 1, never quite reaching them (output is *always* a proper probability). Right panel is the derivative, which peaks at 0.25 when $z = 0$ and vanishes at both extremes — the source of the “vanishing gradient” problem for very deep networks.

⚠ Watch out

When $|z|$ is large (say $z > 6$ or $z < -6$), $\sigma'(z) \approx 0$. Gradients stop flowing through the neuron. For hidden layers this causes the vanishing gradient problem; that is why hidden layers use ReLU instead of sigmoid. For the *output* layer of a binary classifier, sigmoid is still correct.

✓ Key takeaway

$\sigma(z) = 1/(1+e^{-z}) \in (0, 1)$. At $z=0$: output = 0.5 (maximum uncertainty). Derivative = $\sigma(z)(1-\sigma(z))$ peaks at 0.25.

4.2 Logistic regression model

Logistic regression = linear model + sigmoid activation:

$$z = \mathbf{w}^\top \mathbf{x} = w_0 + w_1 x_1 + \dots + w_d x_d, \quad (3)$$

$$\hat{y} = \sigma(z) = P(y = 1 \mid \mathbf{x}; \mathbf{w}). \quad (4)$$

The output $\hat{y}$ is the model’s estimate of the probability that the example belongs to class 1. The decision rule: predict class 1 if $\hat{y} \geq 0.5$ (equivalently, if $z \geq 0$), class 0 otherwise. The **decision boundary** is the hyperplane $\mathbf{w}^\top \mathbf{x} = 0$, which is linear in the feature space.

4.3 Binary cross-entropy loss

MSE is the wrong loss for probabilities — it is designed for continuous outputs. For classification we use the **binary cross-entropy** (BCE) loss, derived from maximum likelihood:

$$\ell(\mathbf{w}; \mathbf{x}, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]. \quad (5)$$

When $y = 1$: loss = $-\log \hat{y}$ (low if $\hat{y}$ is close to 1, huge if close to 0). When $y = 0$: loss = $-\log(1 - \hat{y})$ (low if $\hat{y}$ is close to 0, huge if close to 1). Averaged over all N examples:

$$J(\mathbf{w}) = -\frac{1}{N} \sum_{i=1}^N [y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]. \quad (6)$$

Why cross-entropy over MSE for classification?

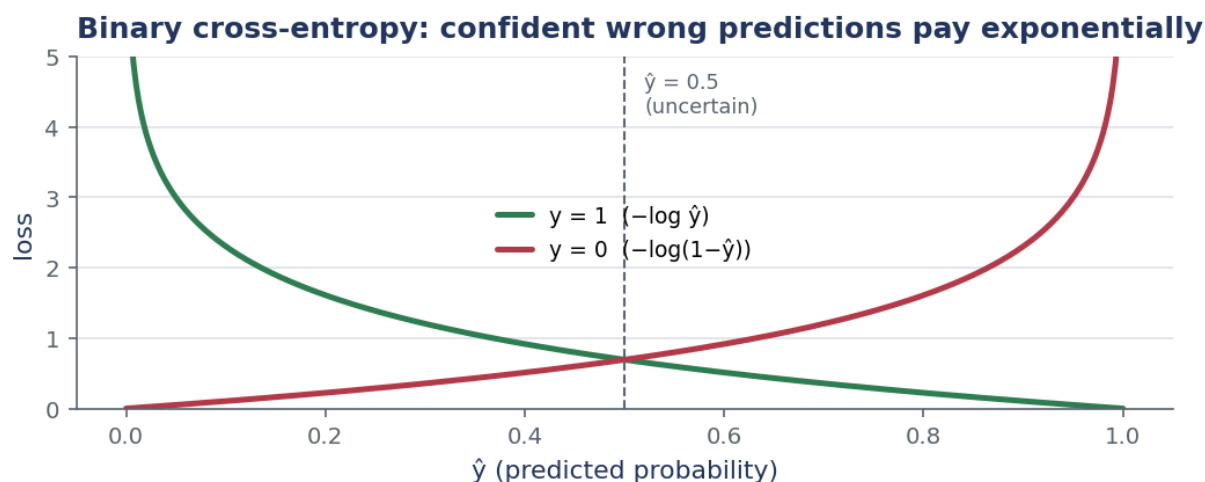


Figure 2: How to read it: green curve ($y = 1$) — loss is low when $\hat{y}$ is near 1 and explodes as $\hat{y} \rightarrow 0$. Red curve ($y = 0$) — the mirror image. The cross-entropy punishes *confident wrong* predictions with near-infinite loss, providing a much sharper learning signal than MSE.

- **Probabilistic:** derived from maximum likelihood estimation — it is the natural measure of how “surprised” the model is by the true label.
- **Exponential penalty:** confident wrong answers pay a huge price, pushing the model hard toward the correct probability.
- **Convex** for logistic regression: guarantees one global minimum.
- **Well-behaved gradient:** the gradient of BCE with sigmoid simplifies beautifully to $(\hat{y} - y)\mathbf{x}$, exactly the same form as in linear regression — just with a different activation in the forward pass.

Worked example: binary cross-entropy by hand

Three examples: $(y = 1, \hat{y} = 0.9)$, $(y = 0, \hat{y} = 0.2)$, $(y = 1, \hat{y} = 0.4)$.

Step 1. Per-example losses.

$$\ell_1 = -(1 \cdot \log 0.9 + 0 \cdot \log 0.1) = -\log 0.9 = \mathbf{0.1054}.$$

$$\ell_2 = -(0 \cdot \log 0.2 + 1 \cdot \log 0.8) = -\log 0.8 = \mathbf{0.2231}.$$

$$\ell_3 = -(1 \cdot \log 0.4 + 0 \cdot \log 0.6) = -\log 0.4 = \mathbf{0.9163}.$$

Step 2. Average BCE. $J = (0.1054 + 0.2231 + 0.9163)/3 = 1.2448/3 = \mathbf{0.4149}$.

Step 3. Sense-check. Example 3 ($y = 1, \hat{y} = 0.4$) contributes most — the model was wrong-ish with moderate confidence. Example 1 ($y = 1, \hat{y} = 0.9$) is nearly right and contributes little.

Key takeaway

Binary cross-entropy $= -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$. Penalises confident wrong predictions exponentially. The standard loss for binary classification.

5 Stochastic gradient descent (SGD)

Session 3 used **batch gradient descent**: compute the gradient over *all* N training examples, then update. For large datasets this is prohibitively slow. **Stochastic gradient descent (SGD)** uses the gradient from a *single* randomly chosen example at each step:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla \ell(\mathbf{w}^{(t)}; \mathbf{x}^{(i)}, y^{(i)}), \quad (7)$$

where (i) is chosen at random. The gradient of BCE for one example is:

$$\nabla \ell = (\hat{y}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}. \quad (8)$$

★ Everyday picture

A bakery owner wants to improve her croissant recipe. Batch GD: she bakes a full day's production (1,000 croissants), gets feedback from every customer, averages it, then adjusts once. SGD: she bakes one croissant, gets one customer's instant opinion, and adjusts the recipe immediately before the next croissant. SGD is noisier but she learns and adapts much faster.

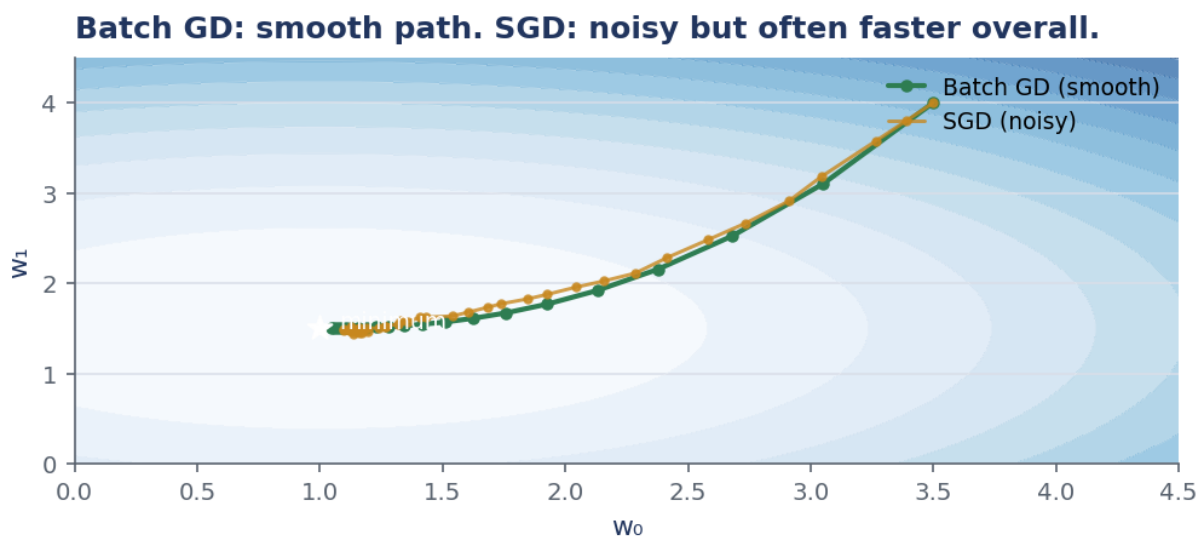


Figure 3: How to read it: green path (batch GD) glides smoothly toward the minimum; amber path (SGD) zigzags noisily but reaches the same neighbourhood much faster. In practice, both converge to a good solution.

Benefits of SGD: faster per iteration (one example, not N); memory efficient (can handle datasets too large to fit in RAM); noise helps escape sharp minima (regularisation effect). **Challenges:** noisy updates mean the loss curve is jagged; convergence near the minimum requires a decaying learning rate.

🔗 Worked example: one SGD step for logistic regression

Setup (from slides): feature $\mathbf{x} = [1, 2, 3]^\top$ (with bias: $[1, 1, 2, 3]^\top$), label $y = 1$, weights $\mathbf{w} = [0, 0, 0, 0]^\top$, $\eta = 0.1$.

Step 1. Forward pass. $z = \mathbf{w}^\top \mathbf{x} = 0$; $\hat{y} = \sigma(0) = 0.5$.

Step 2. Loss. $\ell = -\log(0.5) = \ln 2 \approx 0.6931$.

Step 3. Gradient. $\nabla \ell = (\hat{y} - y)\mathbf{x} = (0.5 - 1)[1, 1, 2, 3]^\top = [-0.5, -0.5, -1.0, -1.5]^\top$.

Step 4. Update. $\mathbf{w} \leftarrow \mathbf{0} - 0.1 \times [-0.5, -0.5, -1.0, -1.5]^\top = [0.05, 0.05, 0.10, 0.15]^\top$.

Step 5. New prediction. $z = 0.05 + 0.05(1) + 0.10(2) + 0.15(3) = 0.05 + 0.05 + 0.20 + 0.45 = 0.75$.
 $\hat{y} = \sigma(0.75) \approx \mathbf{0.679}$. Better than 0.5!

Step 6. Sense-check. The gradient had negative components (because $\hat{y} < y$), so we increased all weights. The model now predicts a higher probability for this positive example.

✓ Key takeaway

SGD update: $\mathbf{w} \leftarrow \mathbf{w} - \eta(\hat{y} - y)\mathbf{x}$ for one example. Fast, memory-efficient, and the noise often helps generalisation.

6 Classification evaluation metrics

Unlike regression (where MSE tells the whole story), classification needs several metrics because different errors have different costs.

The **confusion matrix** tabulates all four prediction outcomes for binary classification:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

From these four numbers, four metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (9)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (10)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (11)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (12)$$

🗣 The intuition

Precision: “of all the times I shouted ‘positive!’, how often was I right?” Recall: “of all the actual positives in the world, how many did I catch?” F1 balances both; it is low if either is low. Accuracy is misleading when one class is rare.

★ Everyday picture

Disease screening: **recall** matters most — a missed sick patient (FN) is dangerous. Spam filtering: **precision** matters most — incorrectly blocking a legitimate email (FP) annoys the user. You can rarely optimise both simultaneously; that trade-off is captured by the F1 score.

📎 Worked example: computing all four metrics from a confusion matrix

Given: $TP = 40$, $FP = 10$, $FN = 5$, $TN = 45$.

Step 1. Total: $N = 40 + 10 + 5 + 45 = 100$.

Step 2. Accuracy: $(40 + 45)/100 = \mathbf{0.85}$ (85% correct).

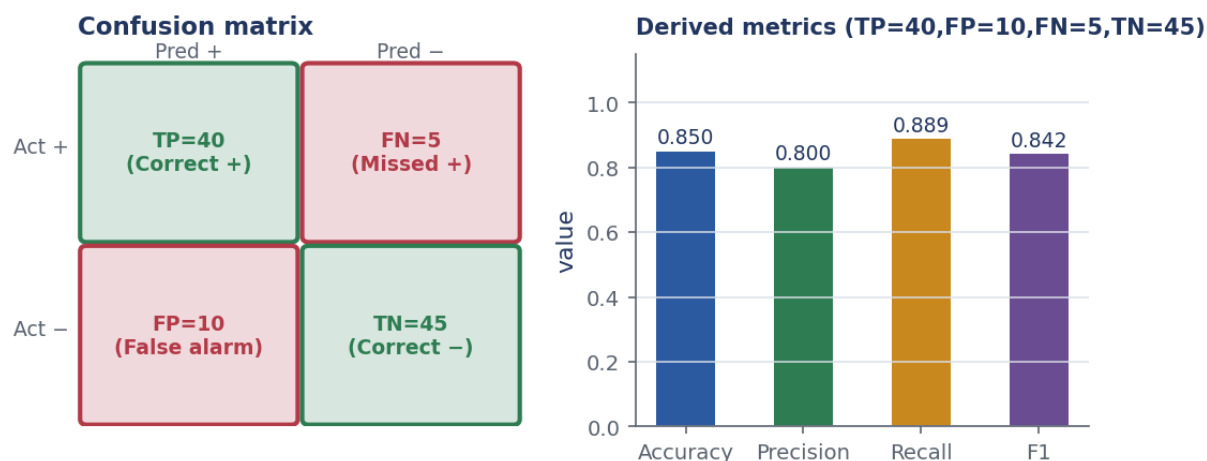


Figure 4: How to read it: left panel is the confusion matrix (green = correct, red = errors). Right panel translates the four cells into the four standard metrics; all are between 0 and 1, higher is better.

Step 3. Precision: $40/(40 + 10) = 40/50 = \mathbf{0.80}$ (80% of predicted positives were real).

Step 4. Recall: $40/(40 + 5) = 40/45 \approx \mathbf{0.889}$ (88.9% of actual positives were caught).

Step 5. F1: $2 \times (0.80 \times 0.889) / (0.80 + 0.889) = 2 \times 0.711 / 1.689 \approx \mathbf{0.842}$.

Step 6. Sense-check. F1 (0.842) lies between precision (0.80) and recall (0.889) — the harmonic mean is always between the two.

✗ Watch out

Accuracy is misleading with imbalanced classes. A dataset with 95% negatives is “solved” by a model that always predicts negative — 95% accuracy, zero recall. Always examine precision, recall, and F1 alongside accuracy.

✓ Key takeaway

Four metrics, one table: accuracy (overall), precision (no false alarms), recall (no misses), F1 (balance). Use recall when FN is costly (medicine), precision when FP is costly (spam filters).

7 Multi-class classification

7.1 From binary to K classes

When $K > 2$, we need K output neurons — one per class — and a way to turn their raw outputs (**logits**) into a valid probability distribution.

One-hot encoding. The label for class k is encoded as a binary vector $\mathbf{y} \in \{0, 1\}^K$ with a 1 in position k and 0s elsewhere:

$$\text{class 3 of 4: } \mathbf{y} = [0, 0, 1, 0]^\top. \quad (13)$$

Why one-hot? It treats all classes equally (no implicit ordering like using $\mathbf{y} \in \{1, 2, 3, 4\}$ would imply), matches the softmax output shape, and simplifies loss computation.

Weight matrix. With K classes and $d + 1$ parameters per class:

$$\mathbf{W} \in \mathbb{R}^{(d+1) \times K}, \quad \mathbf{z} = \mathbf{W}^\top \mathbf{x} \in \mathbb{R}^K. \quad (14)$$

Each column of W is the weight vector for one class. The logit vector $\mathbf{z}$ has one raw score per class.

7.2 Softmax activation

The **softmax** function turns the K logits into a proper probability distribution:

$$\hat{y}_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K. \quad (15)$$

All outputs are positive (exponential of anything is positive), and they sum to 1 (we divide by the total), so they form a valid probability distribution.

The intuition

Softmax is a “smooth argmax”: the class with the largest logit gets the largest share, but all classes get a non-zero probability. Turning up the “temperature” (scaling logits by a small number) makes the distribution flatter; turning it down sharpens it toward a one-hot distribution.

Softmax: $e^{z_k} / \sum e^{z_j}$ converts logits to a probability distribution

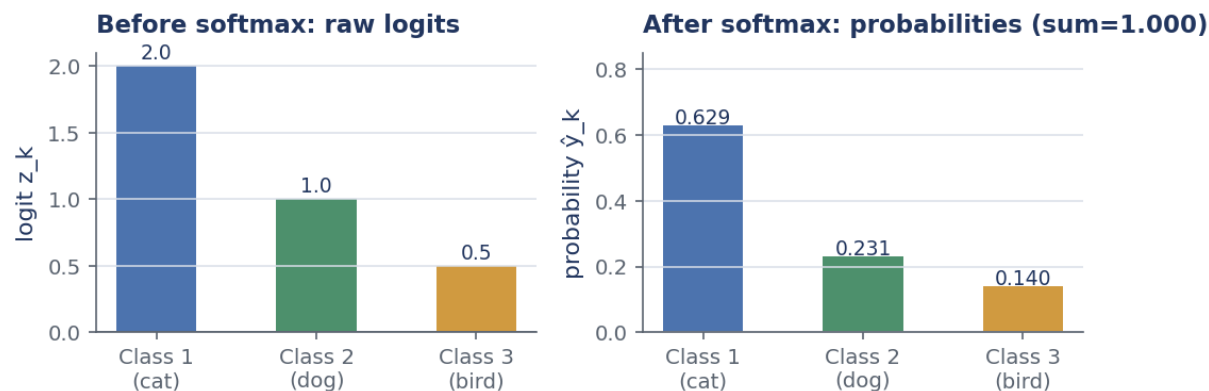


Figure 5: How to read it: left panel — raw logits (any real number). Right panel — after softmax all bars are positive and sum to 1.000 (a probability distribution). Class 1 keeps the biggest share because it had the largest logit.

Worked example: softmax by hand — the slide’s numerical example

Three-class problem. Logits $\mathbf{z} = [2.0, 1.0, 0.5]^\top$.

Step 1. Exponentiate. $e^{2.0} = 7.389$, $e^{1.0} = 2.718$, $e^{0.5} = 1.649$.

Step 2. Sum. $S = 7.389 + 2.718 + 1.649 = 11.756$.

Step 3. Divide each by S .

$$\hat{y}_1 = 7.389 / 11.756 = \mathbf{0.629},$$

$$\hat{y}_2 = 2.718 / 11.756 = \mathbf{0.231},$$

$$\hat{y}_3 = 1.649 / 11.756 = \mathbf{0.140}.$$

Step 4. Check. $0.629 + 0.231 + 0.140 = 1.000$. ✓

Step 5. Predict. Highest probability is class 1 (0.629). The model predicts class 1 with 62.9% confidence.

7.3 Categorical cross-entropy loss

The loss for one example with true one-hot label $\mathbf{y}$ and predicted probabilities $\hat{\mathbf{y}}$:

$$\ell = - \sum_{k=1}^K y_k \log \hat{y}_k. \quad (16)$$

Since $\mathbf{y}$ is one-hot (all zeros except one 1 at position c for the correct class), this reduces to $\ell = -\log \hat{y}_c$ — just the negative log of the correct class's predicted probability. The model is punished for assigning a low probability to the right answer.

Worked example: categorical cross-entropy for the softmax example

True class = 1, so $\mathbf{y} = [1, 0, 0]^\top$. Predictions from above: $\hat{y}_1 = 0.629$, $\hat{y}_2 = 0.231$, $\hat{y}_3 = 0.140$.

Step 1. Loss. $\ell = -(1 \cdot \log 0.629 + 0 \cdot \log 0.231 + 0 \cdot \log 0.140) = -\log 0.629 = \mathbf{0.464}$.

Step 2. Sense-check. The model gave the correct class 62.9% probability. Loss = 0.464. If it had given 100%, loss = 0. If 1%, loss = 4.6. As the model becomes more confident and correct, the loss approaches zero.

Key takeaway

Categorical cross-entropy: $\ell = -\log \hat{y}_c$ (negative log of the correct class's probability). High confidence on the right class $\rightarrow$ low loss.

8 Mini-batch stochastic gradient descent

Batch GD (all N examples) is too slow for large datasets. Pure SGD (1 example) is noisy. **Mini-batch SGD** strikes the balance: randomly sample a **batch** of B examples at each iteration, compute the gradient on those B examples, update:

$$\nabla J_B = \frac{1}{B} \sum_{i \in \text{batch}} \nabla \ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}), \quad \mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J_B. \quad (17)$$

Typical batch sizes: $B \in \{32, 64, 128, 256\}$. Choosing B :

Batch size	Gradient noise	GPU utilisation	Updates per epoch
Small (32–64)	High	Poor	Many
Medium (128–256)	Moderate	Good	Moderate
Large (512–1024)	Low	Excellent	Few

Everyday picture

A restaurant chain samples 50 customers (mini-batch) to get feedback on a new dish. Sampling all 50,000 loyalty-card holders (batch GD) gives more accurate feedback but takes weeks; asking one random customer (SGD) is instant but unreliable. 50 customers gives a good estimate fast.

Why mini-batch SGD? (1) Vectorised matrix multiplies over B examples at once are GPU-friendly — much faster than B separate forward passes. (2) The gradient noise helps escape sharp minima, improving generalisation. (3) The dataset can exceed RAM because you only load B examples at a time.

✓ Key takeaway

Mini-batch SGD: compute gradient on B random examples, update, repeat. Standard in modern deep learning. Typical $B = 32$ –256.

9 Multi-class evaluation: confusion matrix, per-class metrics, top- K

For K classes, the confusion matrix is $K \times K$. The diagonal holds correct predictions; off-diagonal entries are misclassifications. The slide's example (3 classes, 150 examples):

	Pred 1	Pred 2	Pred 3
Actual 1	45	3	2
Actual 2	7	38	5
Actual 3	2	4	44

Accuracy: $(45 + 38 + 44)/150 = 127/150 = 84.7\%$.

Per-class metrics. For class k : TP_k = correct class- k predictions; FP_k = examples of other classes predicted as k ; FN_k = class- k examples predicted as something else.

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \quad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k}, \quad F1_k = 2 \cdot \frac{P_k R_k}{P_k + R_k}. \quad (18)$$

Averaging strategies:

- **Macro average:** average each metric equally across all classes. Treats each class the same, regardless of how many examples it has.
- **Micro average:** pool TP , FP , FN across all classes, then compute. Dominated by frequent classes.
- **Weighted average:** weight each class by its support (number of examples). Standard in scikit-learn's `classification_report`.

Top- K accuracy. For problems with many classes (e.g. ImageNet: 1,000 classes), the model is considered correct if the true class appears among its K most confident predictions. Top-5 accuracy is the standard ImageNet benchmark because even humans sometimes disagree on the “right” label.

✓ Key takeaway

Multi-class evaluation: use the confusion matrix to spot which classes are confused with which. Report per-class precision/recall/F1 and the averaging strategy used. For large K , add top- K accuracy.

10 Recap and what comes next

One paragraph to carry away. Classification predicts a discrete label. Binary: sigmoid activation + binary cross-entropy + SGD. Multi-class: softmax activation + categorical cross-entropy + mini-batch SGD + one-hot encoding. The gradient of cross-entropy with softmax/sigmoid has the elegant form $(\hat{y} - y)\mathbf{x}$ (or

its matrix version). Evaluate with confusion matrix, accuracy, precision, recall, F1, and (for large K) top- K .

✓ Key takeaway

Regression $\rightarrow$ Classification: identity $\rightarrow$ sigmoid/softmax, MSE $\rightarrow$ cross-entropy. Everything else (design matrix, gradient descent, evaluation split) stays the same. This pipeline extends directly to deep networks: just add hidden layers with non-linear activations before the output. That is Session 5.

Further reading. Zhang et al. *Dive into Deep Learning*, Chapter 4; Goodfellow et al., Chapter 6; Bishop, Chapter 4; Murphy, Chapter 10.